Proceedings of 11th International Conference on Computer and Information Technology (ICCIT 2008)
25-27 December, 2008, Khulna, Bangladesh

On the Minimization of Complete Test Set of Reversible *k*-CNOT Circuits for Stuck-at Fault Model

Muhammad Ibrahim, Ahsan Raja Chowdhury and Hafiz Md. Hasan Babu
Department of Computer Science and Engineering
University of Dhaka, Dhaka-1000, Bangladesh
E-mail: ibrahimmuhammad313@yahoo.com, farhan717@univdhaka.edu, hafizbabu@hotmail.com

ABSTRACT — — **In this paper, we propose an algorithm that produces the Complete Test Set (CTS) of a reversible circuit for Single Stuck-at Fault (SSF) and Multiple-Stuck-at Fault (MSF) models. Our algorithm works only for an important subclass of reversible circuits – the circuits consisting of k-CNOT gates (k ≥ 2) though, any n-wire circuit having 0-CNOT or 1-CNOT gates can be converted to a (n + 2) wire circuit having only k-CNOT gates with k ≥ 2 with some additional hardware cost. Generated complete test set is not necessarily optimal, but minimizing the size of the complete test set is our key concern. Finally we provide some experimental results for the proposed method and compare it with existing methods to show how it outperforms almost all of the existing algorithms in terms of number of elements of CTS but is outperformed by some of the existing ones in terms of hardware cost.**

Index Terms — **Automatic Test Pattern Generator (Atpg), Complete Test Set (CTS), Design for Testability (DFT), K-CNOT Circuit, Multiple Stuck at Fault (MSF), Single Stuck at Fault (SSF), Test Vector (TV).**

I. INTRODUCTION

The more logic elements are packed in conventional computer, the more heat is dissipated. When a computational system erases a bit of information, it must dissipate ln 2 x kT energy, where k is Boltzmann's constant and T is the temperature. In room temperature, this is about 2.9 x 10^{-21} joules [1, 2]. Reversible circuits dissipate very low power [3].

A test set is a set of test vectors that are applied to the circuit in question to detect faults, if any. Generating efficient test sets is hard for conventional irreversible circuits though, it is significantly simpler for the case of reversible circuits.

Research reported so far in the field of reversible circuit testing regarding the topic in question is not bountiful. Patel et al. [4] formulated the problem of constructing minimal test set as an Integer Linear Program (ILP) with binary variables. They [4] also used an algorithm that simplifies the aforementioned method for a large circuit by breaking it into smaller sub-circuits [4]. Chakraborty [5] showed that all stuck-at faults in an n-wire k-CNOT circuit $C(n)$ with $k \geq 2$ can be detected using at most n test vectors (TV), namely, n weight-1 vectors. $C(n)$ has a complete test set (CTS) of size n for multiple stuck-at faults (MSF). He [5] proved this using *fixing* property. He has also shown that if an n-wire k-CNOT circuit ($k \geq 0$) is modified by introducing an extra wire that feeds a new control input added to every k-CNOT gate, the resulting circuit has CTS of size 3 for MSFs. Allen et al. [6] proposed an algorithm that detect and localize all Missing Gate Fault (MGF), Lock Gate Fault (LGF) and Single Stuck-at Fault (SSF). Ito et al. [7] showed that it is NP-hard to generate a minimum complete test set for stuck-at faults on a set of wires of a reversible circuit.

We propose a method which exploits *fixing property* of digital circuit, like [5], to produce CTS for a given circuit. We apply a test vector at input of the circuit and observe if the output is same as the input. If it does, then we proceed to the next test vector. If it does not, then some wire(s) of the circuit must be faulty. Our target here is to reduce the size of the complete test set as less as possible without modifying the original k-CNOT circuit with $k \geq 2$. We do not consider fault diagnosis in this paper.

II. ASSUMPTION

We follow the following assumptions throughout the whole paper if otherwise not stated:

a) An n-wire reversible circuit consisting of m number of k-CNOT gates is given ($k \geq 2$ and $m \geq 0$). If we have 0-CNOT and/or 1-CNOT gates in the original circuit, then a design-for-test (DFT) of the circuit can be constructed easily by adding at most 2 extra wires (so that 1-CNOT and 0-CNOT gates becomes 2-CNOT gates having necessary extra control input(s) from the extra wire(s)), so number of wires becomes $n + 2$ and all the methods are applicable on the DFT. Thus, without losing any generality, we assume that we have only k-CNOT gates with $k \geq 2$. When we used any k-CNOT gates with $k = 0$ or 1, we specify explicitly.

b) Let any test vector (TV) be $t_1\ t_2 \ldots t_n$ where t_i is i^{th} bit.

c) In spite of altering any bit in any level of the circuit except the last, if the final output of the circuit is equal to the input, then the input vector is *fixed* by definition. But this concept leads to some extra complexity. So throughout the whole paper, when we say a vector is *fixed* by a circuit, we mean that no bit of the input vector is altered by any gate across the whole circuit.

d) The whole paper is *fixing property* oriented unless otherwise stated. For example, if we say a test vector is

1-4244-2136-7/08/$20.00 ©2008 IEEE

invalid for a circuit, it tacitly implies that this test vector is not *fixed* by that circuit and vice versa.

III. PROPOSED METHOD

A. Description of a Gate and Description Table of a k-CNOT Circuit

Let a gate g has wire indices $g_1, g_2, \ldots, g_p$ as control inputs. Then we call the sequence $g_1, g_2, \ldots, g_p$ the description of the gate g. If g is 0-CNOT gate, then it has no description as we are concerned here with only the control inputs. If we construct a table where the rows are descriptions of the gates of a circuit, then we call this data structure "**Description Table**" of the circuit.

Example 3.1. Fig. 3.1(b) shows the description table of the circuit shown in Fig. 3.1(a).

B. Algorithm

We will use Description table, Status table and $TV_{(temp)}$ (Temporary Test Vector). Description table is already defined. The Status table is an n x 2 array. Each row holds the current test status for both SA-0 and SA-1 faults of ith wire. (For example, 0 if not tested, 1 if tested) $TV_{(temp)}$ is an n element array. This is the temporary TV of which our algorithm converts some of its '1' bits to 0.

Procedure 1: **Process_description_table**

Input: *Description table* of a circuit.
Output: *Description table* of the circuit with some sorting applied (the type of sorting is discussed later).
Step 1: Sort each row of the description table in increasing order of their elements.
Step 2: Treat a row of a description table as a single element and then sort all rows in decreasing order of their first element.
return *Description table.*

Procedure 2: **Adjust_status_table**

Input: *Status table and a TV*
Output: *Adjusted status table.*
Step 1: Adjust the status table as follows:
for all i in $\{1, 2, \ldots, n\}$, **do**
if i^{th} bit of TV is 0, then set 1st element of ith row of the status table to 1.
else set 1st element of ith row of the status table to 0.
return *Status table.*

Procedure 3: **Get_New_TV**

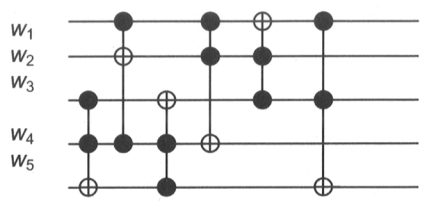
Input: *Status table.*
Output: *A TV that acts as the $TV_{(temp)}$ for the next iteration.*
Step 1: Calculate the TV as follows:
for all i in $\{1, 2, \ldots, n\}$, **do**
if i^{th} row of *status table* has 0 in the second element, set t_i to 1.
else set t_i to 0.
return TV.

Algorithm 1: **Efficient_Complete_Test_Set_Generator (ECTSG)**

Input: The configuration of a k-CNOT ($k \geq 2$) circuit is in *description table* as input. Input pattern is as follows: each gate occupies 1 row in the *description table*. The elements of each row are the wire indices of a particular gate where this gate has control inputs. Inversion input wire index is not entered in the row. Each row is terminated with a -1 value which works as sentinel.
Output: A set CTS of TVs that constitute the CTS for the given circuit. CTS is initially empty.
Step 1: **(a)** Initialize $TV_{(temp)}$ with all 1 i.e. 1^n.
(b) Initialize status table by filling all n entries of its both columns with 0. First and second columns represent SA-1 and SA-0 fault respectively.
Step 2: call *Process_description_table.*
Step3: **for** each gate g in *description table* **do**
if g does not fix $TV_{(temp)}$, **then do**
let w be the first wire index of this gate in description table i.e. first element of the row meant for this gate. Set t_w in $TV_{(temp)}$ to 0.
Step 4: Now after possible modification, $TV_{(temp)}$ is a valid TV of CTS. So include it in the set CTS.
Step 5: call *Adjust_status_table* with $TV_{(temp)}$ as its argument.
Step 6: Check if status table is full or not i.e. if there is any element in any row is 0 still, indicating that this wire needs further testing. If no such element found, then the algorithm halts at this point.
Step 7: If the algorithm does not halt in step 6, then it indicates that there exist some i such that i^{th} wire is not tested for SA-0 and/or SA-1 fault. So we need at least one more TV.
(a) Call *Get_New_TV*$_{(temp)}$. The retuned TV by *Get_New_TV*$_{(temp)}$ is the $TV_{(temp)}$ for the next iteration.
(b) Go to step 3.
return CTS.

The following example demonstrates how ECTSG works.

Example 3.2: Consider a circuit whose description table contains the rows (4, 5), (3, 4), (2, 3), (1, 4), (1, 3) and (1,2)



(a)

Gate	Indices of Control Input	
1	3	4
2	1	4
3	4	5
4	1	2
5	2	3
6	1	3

(b)

Fig. 3.1 (a) Original circuit (b) Corresponding description table

respectively, after sorting i.e. after calling the *process_description_table* procedure.

We have shown some initial tables maintained by ECTSG in Fig. 3.2 and 3.3. The reader can verify that ECTSG generates 3 TVs as CTS for this circuit: TV_1 = 0 0 1 0 1, TV_2 = 0 1 0 1 0, TV_3 = 1 0 0 0 0; hence |CTS| = 3. □

Wire index	Tested for SA-1 fault (0 if not tested, 1 otherwise)	Tested for SA-0 fault (0 if not tested, 1 otherwise)
1	0	0
2	0	0
3	0	0
4	0	0
5	0	0

$TV_{1(temp)}$	TV_1
1	0
1	0
1	1
1	0
1	1

(a)

Wire index	Tested for SA-1 fault	Tested for SA-0 fault
1	1	0
2	1	0
3	0	1
4	1	0
5	0	1

(b)

Fig. 3.3 (a) Test Vector (TV_1) after **Pass 1** (b) Adjusted *status table* for TV_1's effect

If no sorting is performed on the *description table* i.e. if *Process_description_table* is not called in step 2 of ECTSG, the correctness of the proposed method will not be hampered but possibly different CTS will be yielded. What is the rationale for sorting then? To be convinced, consider the following arguments.

In the first place, we sort each row of *description_table* in increasing order of its elements. Now we will investigate the effect if the rows of *description_table* are sorted by *description_table[i][1]*. Consider the *description table* of Fig. 3.4(b) and Fig. 3.4(c) where the rows of *description_table* are sorted by increasing and decreasing order of the *description_table[i][1]* respectively ($1 \le i \le m$). Fig. 3.4(a) shows the original *description table*.

Now we assume that ECTSG does not call *Process_description_table* in Step 2 rather works directly on the input *description table*. Table I shows that the sorting style adopted in Fig. 3.4 (c) yields most efficient result i.e. |CTS| = 3 that is less than the other two.

The reason is: in both cases we choose the first control input k of a gate (i.e. first element of a row in *description_table* i.e. *description_table[i][1]*) to set t_k in $TV_{(temp)}$ to 0 if it is not already 0. In the first case of our example given above, the abovementioned first control input k may appear only in the rows where description_*table[i][1]* is the first element – possibly few because it is not natural to have all the gates connected with 1st wire.

But in the second case, the first control input k may appear throughout all subsequent rows in the *description_table* because the elements of each row are sorted in increasing order, therefore possibly reducing the total number of changes in $TV_{(temp)}$. You can recall here that ECTSG algorithm's basic target is to retain $TV_{(temp)}$ as much as possible in a pass to generate the corresponding actual TV.

We can conclude from the above discussion that ECTSG with sorting according to the discussed style *may* outperform ECTSG without sorting. We say 'may' because we are not providing any rigorous proof of our conjecture. So we are not ruling out the possibility of getting worse result after sorting according to the abovementioned style for a particular circuit. What we have shown using the above examples is based on empirical observations.

C. Evaluation of Proposed Method

i) Optimality of ECTSG

ECTSG algorithm does not necessarily return optimal result but it does in many of the cases. We mentioned earlier that it is NP-hard to generate a minimum complete test set for stuck-at faults on a set of wires of a reversible circuit [7].

Gate	Indices of Control Input Gate	
1	3	4
2	1	4
3	4	5
4	1	2
5	2	3
6	1	3

(a)

Gate	Indices of Control Input Gate	
1	1	2
2	1	3
3	1	4
4	2	3
5	3	4
6	4	5

(b)

Gate	Indices of Control Input Gate	
1	4	5
2	3	4
3	2	3
4	1	2
5	1	4
6	1	3

(c)

Fig. 3.4 (a) Original *description_table* for the sample circuit Fig. 3.1(a), (b) Rows sorted in increasing order of the first elements and each row is also sorted in increasing order (c) Rows sorted in decreasing order of the first elements and each row is also sorted in increasing order.

TABLE I

COMPARISON OF DIFFERENT SORTING STYLES ON *DESCRIPTION TABLES* SHOWN IN FIG. 3.4

Description Table	|CTS|	CTSs
Fig. 3.4 (a)	4	(0 1 0 0 1), (1 0 0 0 0), (0 0 0 1 0), (0 0 1 0 0)
Fig. 3.4 (b)	5	(0 0 0 0 1), (0 0 0 1 0), (0 0 1 0 0), (0 1 0 0 0), (1 0 0 0 0)
Fig. 3.4 (c)	3	(0 0 1 0 1) , (0 1 0 1 0) , (1 0 0 0 0)

ii) Time Complexity Analysis

Theorem 1: Let an n-wire circuit consisting of m number of k-CNOT ($k \geq 2$) gates is given and ${}^nC_2 + {}^nC_3 \ldots + {}^nC_{n-1} = q$; ${}^qP_1 + {}^qP_2 + \ldots + {}^qP_q = p$. ($C$ and P stands for combination and permutation respectively).Then:

(a) There are total p number of different circuits C_i where each k-CNOT gate appears at most once, and then $max\ (m_1, m_2, \ldots, m_p) = q = O\ (2^n)$; m_i is the number of gates in circuit C_i ($1 \leq i \leq p$). (It is important to note that here we treat two gates as distinct only if their control inputs differ; we do not treat two gates as distinct if they have same control inputs but different inversion input. This assumption is inline with our assumptions throughout the whole book because we said earlier that the total book is *fixing* property oriented – so we are not concerned with the inversion input of a k-CNOT gate.)

(b) If we allow more than one occurrence of a gate that have same control input wires in a circuit, then $max\ (m_1, m_2, \ldots, m_p) = \infty$. ■

Proof: (a) As k-CNOT gate has maximum k control inputs and the number of wires is n, so $k \leq (n-1)$. We can have ${}^nC_2$ number of 2-CNOT gates, ${}^nC_3$ number of 3-CNOT gates, …, ${}^nC_{n-1}$ number of $(n-1)$-CNOT gates in a n-wire k-CNOT circuit with $k \geq 2$. So the total number of possible gates are ${}^nC_2 + {}^nC_3 + \ldots + {}^nC_{n-1} = 2^n - n - 2 = q$. Now if we build a circuit where all of the above mentioned gates are used exactly once, then $m = q$. If some of these gates are not used but the used gates have exactly one occurrence as before, then $m < q$. Now we can permute these q gates to get a circuit, so the total number of circuits comprised of these q gates is: ${}^qP_1 + {}^qP_2 + \ldots + {}^qP_q = p$.

(b) If a circuit is allowed to have more than one instance of any gate, then m may be raised to ∞ because we can append as many gates as we wish to build an arbitrary circuit, but use of some of these gates may be redundant.□

Lemma 1: We can confine the size of the *description_table* of ECTSG algorithm to maximum $q = O\ (2^n)$ ($q \neq \infty$). ■

Proof: We impose a condition that same ordering of values of the rows of the *description_table* should be adopted while taking input – for example, if (1, 3) is entered as description of a gate then (3, 1) is considered as an old gate and vice versa as these two descriptions are in fact same from ECTSG's viewpoint. The reasoning is: if the circuit has gates having same wires as control inputs, then they will have identical rows in the *description_table* because we are not storing the index of *inversion* input wire. As we are using these rows to fix the $TV_{(temp)}$ in ECTSG algorithm, once we have checked a row, no need to check another identical row. Hence we can keep only one row in case of duplicity, which completes the proof □

From the above discussion it is clear that ECTSG may not need to check all the m gates of a circuit. As each run of the ECTSG scans all the entries of the *description table*, so the outcome of lemma 1 may save a lot of computation. So ECTSG is computationally feasible.

IV. EXPERIMENTAL RESULT

The result after the implementation of ECTSG is depicted in Table II. The designs of the benchmark circuits are taken from [8]. It is important to note that these machine-readable versions posted on [8] have a

characteristic: all the elements (discarding the *inversion* input of a gate) of a row are sorted in increasing order. So result shown in Table II produced by ECTSG algorithm is based on this structure, but actually ECTSG algorithm not necessarily needs the rows sorted. But the result may be different if different structures are used. Table II also contains a column that shows the result of ECTSG without sorting described in Section 3.2. From this column we notice that our conjecture mentioned in Section 3.2 holds correct for all the benchmark circuits as no result is better than the result found with sorting but converse is true for many cases.

Authors of [5] and [6] also used the benchmark circuits posted in [8]. As we said earlier that [5] showed that the size of CTS can be reduced to only 3 for any circuit consisting of k-CNOT gates ($k \geq 0$) but **always** needs modification of the circuit by adding one extra wire and replacing **every** k-CNOT gate by (k +1)-CNOT gate i.e. **always** needs to construct a DFT. As there may be hundreds of thousands of gates in a real-world circuit, it may be suitable to opt for ECTSG algorithm in this regard, which **may** need to construct DFT by adding maximum two extra wires and to replace **only** the k-CNOT gates having $k = 0$ and 1 by ($k + 2$)-CNOT and ($k + 1$)-CNOT gates respectively

TABLE II
EXPERIMENTAL RESULTS FOR DIFFERENT BENCHMARK CIRCUITS

Benchmark Circuits	Model	Bits of original circuit	Bits of DFT	Number of gates	Number of TVs by ECTSG without sorting*	Number of TVs by proposed ECTSG*	Number of TVs in [5]*	Number of TVs in [6]**
2of5	GT	6	8	18	2	2	8	4
4_49	GT	4	6	16	4	4	6	4
3_17	NCT	3	5	6	3	3	5	4
6sym	NCT	10	11	20	6	4	11	5
9sym	NCT	12	13	28	8	4	13	5
Cycle 10_2	GT	12	13	19	2	2	13	5
Cycle 17_3	GT	20	21	48	2	2	21	6
Gray code function	N/A	6	7	5	2	2	7	N/G
Ham 3	NCT	3	4	5	3	3	4	N/G
Ham7	GT	7	8	23	3	3	8	4
Ham 15	GT	15	16	132	5	5	16	12
Hwb4	GT	4	5	17	4	4	5	4
Hwb5	GT	5	6	55	6	6	6	7
Hwb6	GT	6	7	126	7	7	7	11
Hwb7	GT	7	8	289	8	8	8	13
Hwb8	GT	8	10	637	9	9	10	N/G
Hwb9	GT	9	10	1544	10	9	10	N/G
Hwb10	GT	10	11	3631	11	11	11	N/G
Hwb11	GT	11	12	9314	12	12	12	N/G
Mod5	NCT	5	6	8	3	3	6	N/G
Mod5adder	GT	6	8	21	3	3	8	4
Mod1024adder	GT	20	21	55	3	3	21	N/G
Mod 1048576adder	GT	40	41	210	3	3	41	N/G
Rd32	NCT	4	5	4	3	3	5	3
Rd53	GT	7	9	30	3	3	9	N/G
Rd73	NCT	10	11	20	7	4	11	4
Rd84	NCT	15	16	28	8	4	16	6
Xor5	NCT	5	6	4	2	2	6	N/G

** means this case works on DFT i.e. needs same number of bits as DFT.*
*** means this case works on original circuit i.e. needs same number of bits as original circuit.*
N/G and N/A stands for "Not Given in this paper" and "Not Applicable" respectively.

That means if $k \geq 0$ in the original circuit, then we have to add 2 extra wires and all 0-CNOT and 1-CNOT gates are replaced by 2-CNOT gates, and if $k \geq 1$ in the original circuit, then we have to add 1 extra wire and all 1-CNOT gates are replaced by 2-CNOT gates, and if $k \geq 2$ then no need to construct DFT. Obviously, the less the number of control inputs in k-CNOT gate, the cheaper the gate is [9], so replacing 0-CNOT and/or 1-CNOT gates should not cost comparatively much. Moreover, in our case |CTS| may be 2 i.e. optimal, but minimum |CTS| of [5] is 3 for the same circuit at hand. The other method of [5] generates n TVs, namely, n weight-1 vectors, for n-wire k-CNOT ($k \geq 2$) circuit regardless the number of gates. In Table II we show that the number of TVs of the proposed is better than the method of [5] in most of the cases which generates exactly n TVs as CTS for an n-wire k-CNOT circuit ($k \geq 2$); and in the rest of the cases it generates equal number of TVs as [5].

Some other researchers, specifically, Patel et al. [4] and Allen et al [6] did not use *fixing property*. Both the algorithms of [4] and [6] outperform ECTSG in terms of the need for modification of the original circuit (i.e. hardware cost) as none of their technique requires to construct DFT like ECTSG (possibly i.e. in case of existing 0-CNOT and/or 1-CNOT gates); but as we said earlier that cost of 2-CNOT gates, which ECTSG requires to be added in a circuit where 0-CNOT and/or 1-CNOT gates exist, are nominal [9]. Moreover, ECTSG algorithm is superior to [6] in most cases, shown in Table II in terms of |CTS| and equal in rest of the cases. The computational cost of [4] and [6] is presumably larger than ECTSG. And lastly, in [6], minimum |CTS | is 3 in table II but it is reduced in ECTSG to 2.

V. CONCLUSION

Utilizing fixing property, we presented an algorithm that generates complete test set for stuck-at model for reversible circuits composed of k-CNOT gates with $k \geq 2$. The limitation $k \geq 2$ can be lifted because any n-wire k-CNOT circuit with $k \geq 0$ can be converted to a $(n + 2)$-wire k-CNOT circuit with $k \geq 2$. We also provided some experimental results and comparison between the existing methods.

REFERENCES

[1] C. Bennett, "Logical reversibility of computation", *IBM Journal of Research and Development* 17 (1973) 525–532.

[2] R. Landauer, "Irreversibility and heat generation in the computational process", *IBM Journal of Research Development* 5 (1961) 183–191.

[3] http://www.zyvex.com/nanotech/reversible.html

[4] K.N. Patel, J.P. Hayes, I. L. Markov, "Fault testing for reversible circuits", *VLSI Test Symposium*, 21 (2003) 410–416.

[5] A. Chakraborty, "Synthesis of Reversible Circuits for Testing with Universal Test Set and C-Testability of Reversible Iterative Logic Arrays", *18th International Conference on VLSI Design* (2005) 249-254.

[6] J.S. Allen, Jacob D. Biamonte, Marek A. Perkowski, "ATPG for Reversible Circuits using Technology-Related Fault Models", *International Symposium on Representations and Methodologies for Emergent Computing Technologies*, Tokyo, Japan, September (2005).

[7] I. Shigeru, I. Yusuke, T. Satoshi, U. Shuichi, "On the Complexity of Fault Testing for Reversible Circuits", *IEIC Technical Report (Institute of Electronics, Information and Communication Engineers)* 105 (2005) 13-16.

[8] D. Maslov, G. Dueck, and N. Scott, Reversible Logic Synthesis Benchmarks Page. http://webhome.cs.uvic.ca/~dmaslov/